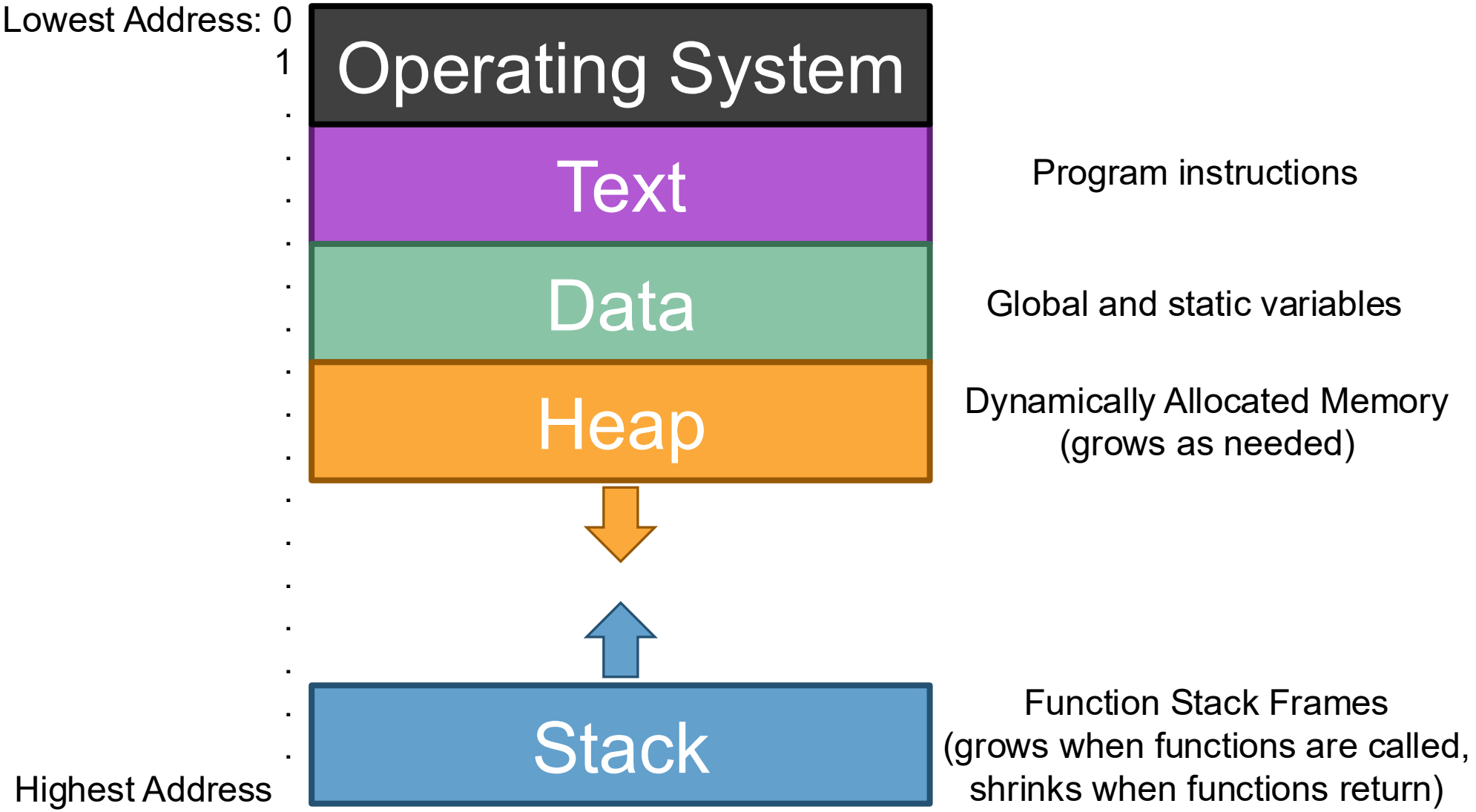


Pointers and the Stack

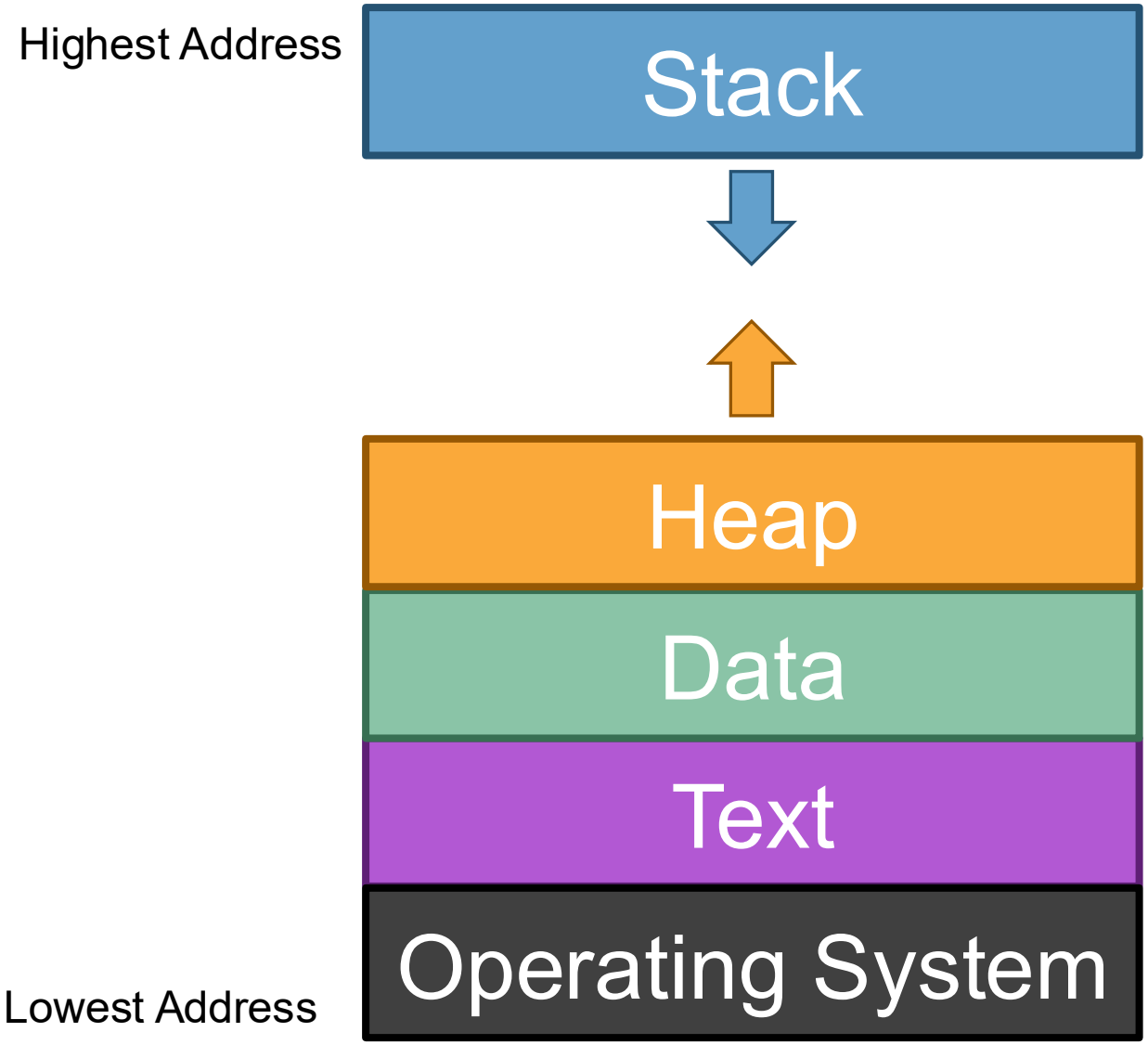
COMP211 Connor McMahon

Parts of Program Memory

Parts of Program Memory



Parts of Program Memory



The Stack (Automatic Memory)

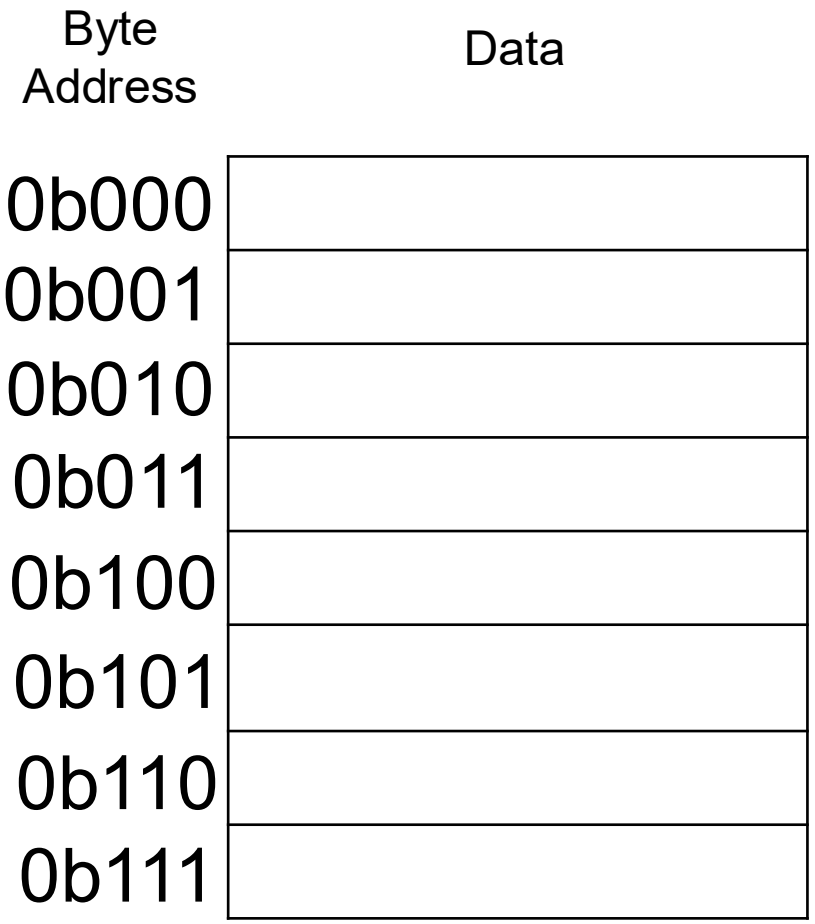
- The stack is used for each function's stack frame
- Stack Growth
  - Every time a function is called, a stack frame is **pushed** onto the stack
  - This is done automatically (the programmer does not have to think about it)
  - Grows from higher memory addresses to lower
- Stack Shrinkage
  - When the function returns, its stack frame is **popped** off the stack
  - This is done automatically (the programmer does not have to think about it)
- **Stack Overflow**
  - If the stack grows too large (e.g., due to deep recursion or very large local variables), you might encounter a stack overflow.

Memory Addresses

Size of Memory Address

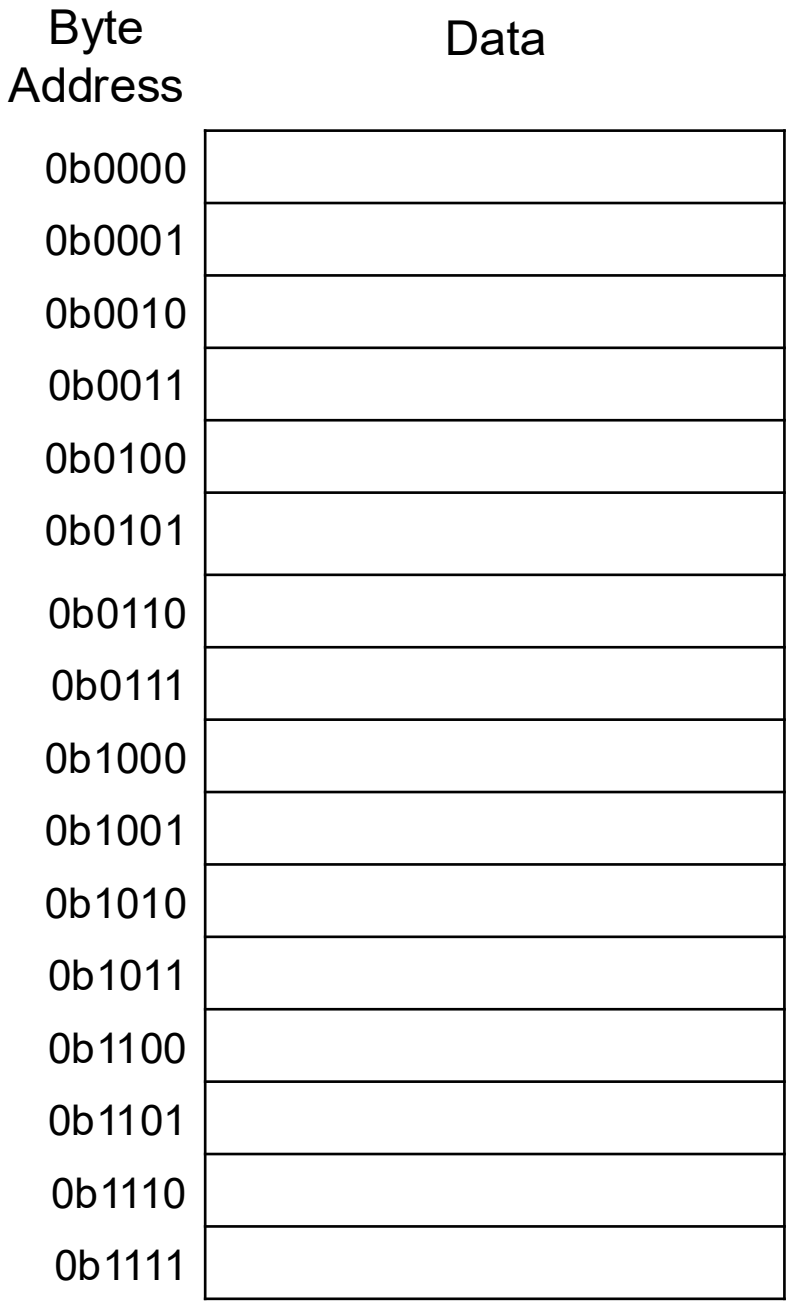
- The size of our memory determines the number of bits we need to address it

- For example, if our memory could only hold 8 bytes, we would need $\log_2(8) = 3$ bits to address the memory



Size of Memory Address

- As another example, if our memory could only hold 16 bytes, we would need $\log_2(16) = 4$ bits to address the memory



Size of Memory Address

- Q1) If the size of our memory is 1 GB, how many bits do we need to address our memory? (1GB = 2^{30} bytes)

- Q2) How (in GB) large can our memory be with a 32-bit address space?

- Q3) How (in GB) large can our memory be with a 64-bit address space?

Modern Processors

- 32-bit architectures dominated from the mid-1980s through the 1990s.

- The transition to 64-bit computing began in the early 2000s and was largely complete by 2010.

- **This is why pointers are 8 bytes!!!**

Modern Processors

- Most 64-bit CPUs use only 48–52 bits of their 64-bit address space.

- Why Not the Full 64 Bits?
  - Hardware and OSes reserve upper bits for future use.
  - 48-bit addressing already supports 256 TB.
  - Plenty of room to grow as physical memory sizes increase.

- With 48 bits, the address space is limited to:
  - 2^{48} bytes
  - $(2^{18} * 2^{30})$ bytes
  - 256 TB

Pointers

What is a pointer

- A **pointer** is a variable that stores a **memory address.**
  - The memory address is the location of the variable in memory.
- A pointer does not store the value! It stores where the value lives.

Why use pointers?

- Pointers allow us to
  - Modify variables in another function
  - Work with dynamic memory
  - Build linked data structures
  - Efficiently pass large data structures

Declaring Pointers

- Syntax
  - `type *name;`
- Examples
  - `int *p;` ← p is a pointer to an int
  - `char *c;` ← c is a pointer to a char
  - `double *d;` ← d is a pointer to a double

Pointer Types Matter

- Do not mix pointer types and data types!!!

Pointer Type	What it can point to
`int8_t *`	Only `int8_t` variables
`int16_t *`	Only `int16_t` variables
`char *`	Only `char` variables
`double *`	Only `double` variables

Getting the Address of a Variable

- The **address-of** operator (&) is used to get the address of a variable

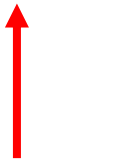
```
int x = 5;
int *p = &x;
```

p is a pointer to an integer

Getting the Address of a Variable

- The **address-of** operator (&) is used to get the address of a variable

```
int x = 5;
int *p = &x;
```



& gets the address of x
(location where x is stored in memory)

Getting the Address of a Variable

- The **address-of** operator (&) is used to get the address of a variable

```
int x = 5;
int *p = &x;
```


The value of p is the address of x
(the location where x is stored in memory).

Dereference Operator

- The dereference operator is used to access the value stored at an address.

```
int x = 5;
int* p = &x;
*p = 10;
```



Now, the value of x is 10.

NULL Pointers

- NULL is used to indicate an invalid or uninitialized pointer
- Never dereference a NULL pointer!!!

```
int *p = NULL;
```

Pointers Summary

- **Pointer:** A variable that holds an address
- Pointer Declaration

```
int x = 10;
int *ptr = &x;
```

  - ptr is a pointer that can store the address of an int.
- Reading a Variable with the Dereference Operator

```
int value = *ptr;
```

  - *ptr accesses (reads) the value stored at the memory address ptr points to.
- Writing to a Variable with the Dereference Operator

```
*ptr = 20
```

  - Changes the value at the memory location ptr points to.

Address Example

```c
#include <stdio.h>

int main() {
    int x = 10;

    int* p = &x; // p stores the address of x

    printf("Value of x: %d\n", x);
    printf("Address of x: %p\n", &x);
    printf("Pointer p holds: %p\n", p);
    printf("Value at address p: %d\n", *p);

    return 0;
}
```

```
Value of x: 10
Address of x: 0xfffffdd26e5c
Pointer p holds: 0xfffffdd26e5c
Value at address p: 10
```

All pointers are 8 bytes.

Example 1

What does this program print?

```c
#include <stdio.h>

int main(void) {
   int x = 5;
   int* p = &x;
   *p = 8;
   printf("%d\n", x);
}
```

Example 1

What does this program print?

```c
#include <stdio.h>

int main(void) {
   int x = 5;
   int* p = &x;
   *p = 8;
   printf("%d\n", x);
}
```

8

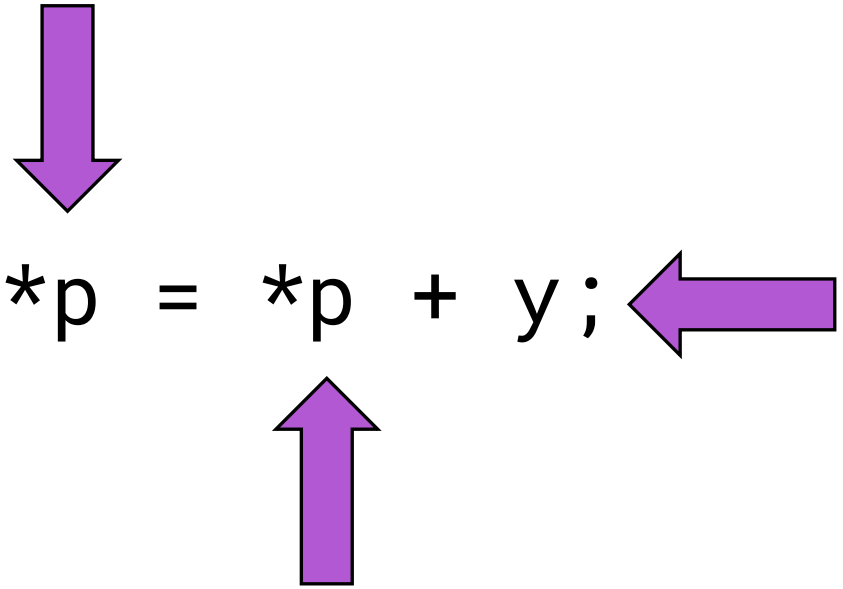
Example 2

```
 1 #include <stdint.h>
 2 #include <stdio.h>
 3
 4 int main(void) {
 5    int32_t x = 1;
 6    int32_t y = 2;
 7    int32_t* p = &x;
 8
 9    *p = *p + y;
10    printf("%d %d\n", x, y);
11 }
```

Memory diagram before `printf` is called

Example 2

Follow the arrow once,
then write to that location.



```
*p = *p + y;
```

Read the value at this location.

Follow the arrow once,
then read the value at
that location.

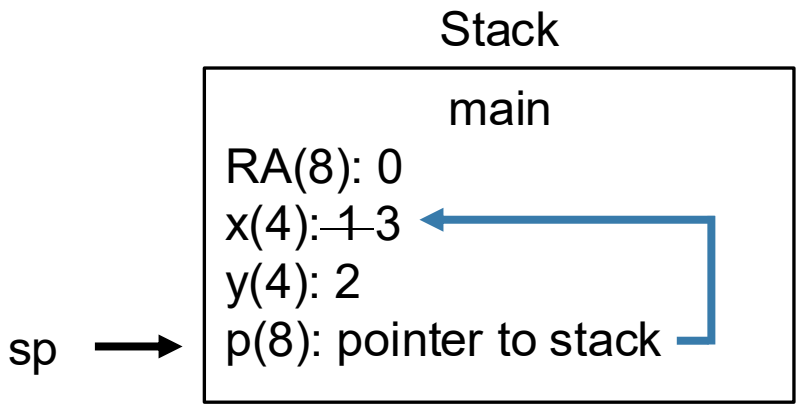
Example 2

```
1  #include <stdint.h>
2  #include <stdio.h>
3
4  int main(void) {
5     int32_t x = 1;
6     int32_t y = 2;
7     int32_t* p = &x;
8
9     *p = *p + y;
10    printf("%d %d\n", x, y);
11 }
```

Program Output

3, 2

Memory diagram before `printf` is called



The value of the pointer is listed as pointer to stack, and an arrow is drawn to the location it points to.

Recall: Stack Pointer

- **Stack Pointer (SP):** A register that points to the current top of the stack (the location where the most recent stack data was stored).

- **Register:** A small, fast storage location inside the CPU that holds data the CPU is currently working with.

- As a program calls functions and uses the stack, the stack pointer changes to indicate where the stack currently ends.

- On many architectures the stack grows toward **lower memory addresses**. So adding data to the stack typically **decreases** the stack pointer, while removing data typically **increases** it.

Example 3

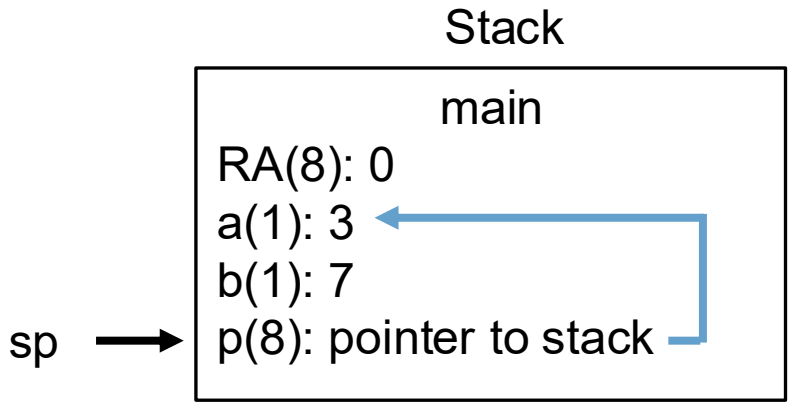
```
1  #include <stdint.h>
2  #include <stdio.h>
3
4  int main(void) {
5     int8_t a = 3;
6     int8_t b = 7;
7     int8_t* p = &a;
8
9     p = &b;
10    *p = 10;
11
12    printf("%d %d\n", a, b);
13 }
```

Memory diagram after line 7 is executed

Example 3

```
1  #include <stdint.h>
2  #include <stdio.h>
3
4  int main(void) {
5     int8_t a = 3;
6     int8_t b = 7;
7     int8_t* p = &a;
8
9     p = &b;
10    *p = 10;
11
12    printf("%d %d\n", a, b);
13 }
```

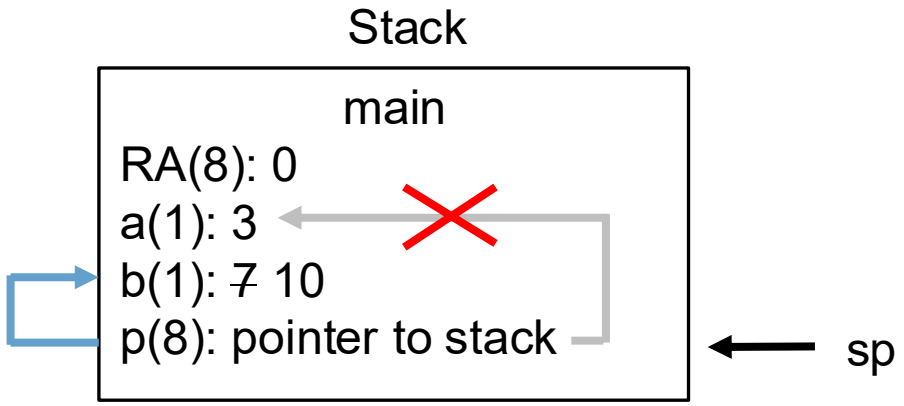
Memory diagram after line 7 is executed



Example 3

```
1  #include <stdint.h>
2  #include <stdio.h>
3
4  int main(void) {
5      int8_t a = 3;
6      int8_t b = 7;
7      int8_t* p = &a;
8
9      p = &b;
10     *p = 10;
11
12     printf("%d %d\n", a, b);
13 }
```

Memory diagram right before `printf` is called



Program Output

3, 10

Example 4

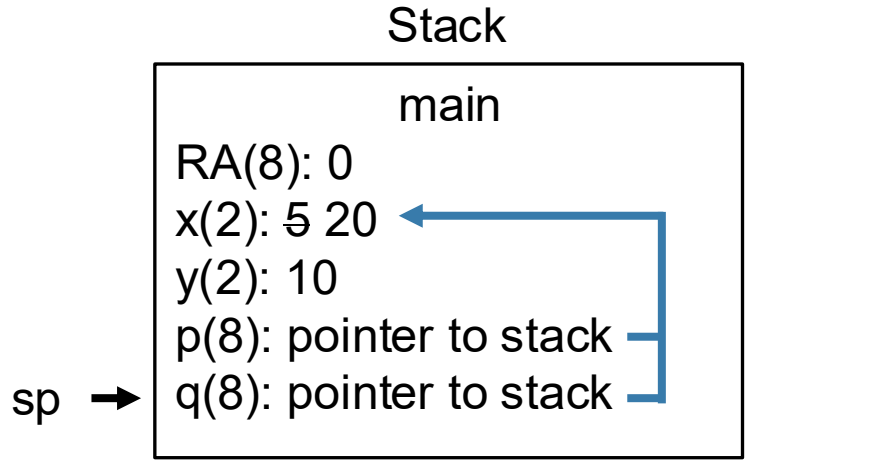
```
1  #include <stdint.h>
2  #include <stdio.h>
3
4  int main(void) {
5      int16_t x = 5;
6      int16_t y = 10;
7      int16_t* p = &x;
8      int16_t* q = p;
9      *q = 20;
10
11     printf("%d, %d, %d, %d\n", x, *p, y, *q);
12 }
```

Memory diagram before `printf` is called

Example 4

```
1  #include <stdint.h>
2  #include <stdio.h>
3
4  int main(void) {
5      int16_t x = 5;
6      int16_t y = 10;
7      int16_t* p = &x;
8      int16_t* q = p;
9      *q = 20;
10
11     printf("%d, %d, %d, %d\n", x, *p, y, *q);
12 }
```

Memory diagram before `printf` is called



Program Output

20, 20, 10, 20

Aliasing

- **Aliasing** occurs when two or more pointers point to the same location.
- In the previous example, p and q are aliases of x.

Pointers

Draw the memory diagram just before printf is executed.
What is the output of the program?

```c
#include <stdint.h>
#include <stdio.h>

int main(void) {
    int64_t x = 4;
    int64_t y = 7;
    int64_t* p = &x;
    int64_t* q = &y;

    *p = *q;
    *q = 10;

    printf("%ld, %ld, %ld, %ld\n", x, *p, y, *q);
}
```

Memory diagram just before `printf` is called

Program Output

Double Pointers

- A double pointer is a pointer to a pointer
  - `int **dp;`
  - `dp` stores the address of an `int *`
  - `*dp` is a pointer to an `int`
  - `**dp` is an `int`

Double Pointer Example

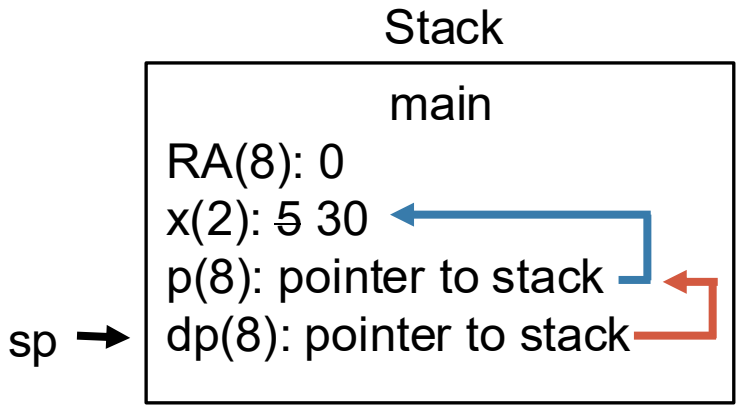
```
1  #include <stdint.h>
2  #include <stdio.h>
3
4  int main(void) {
5     int16_t x = 5;
6     int16_t* p = &x;
7     int16_t** dp = &p;
8     **dp = 30;
9
10    printf("%d %d %d\n", x, *p, **dp);
11 }
```

Memory diagram just before `printf` is called.

Double Pointer Example

```
1  #include <stdint.h>
2  #include <stdio.h>
3
4  int main(void) {
5     int16_t x = 5;
6     int16_t* p = &x;
7     int16_t** dp = &p;
8     **dp = 30;
9
10    printf("%d %d %d\n", x, *p, **dp);
11 }
```

Memory diagram just before `printf` is called.



The program prints
30 30 30

Double Pointers

Follow the arrow twice,
then write to that location.



```
**dp = 30
```

Memory Diagram

```c
#include <stdint.h>
#include <stdio.h>

int main(void) {
    int32_t a = 10;
    int32_t b = 20;

    int32_t *p = &a;
    int32_t *q = &b;
    int32_t **pp = &p;

    *pp = q;
    **pp = *p + a;

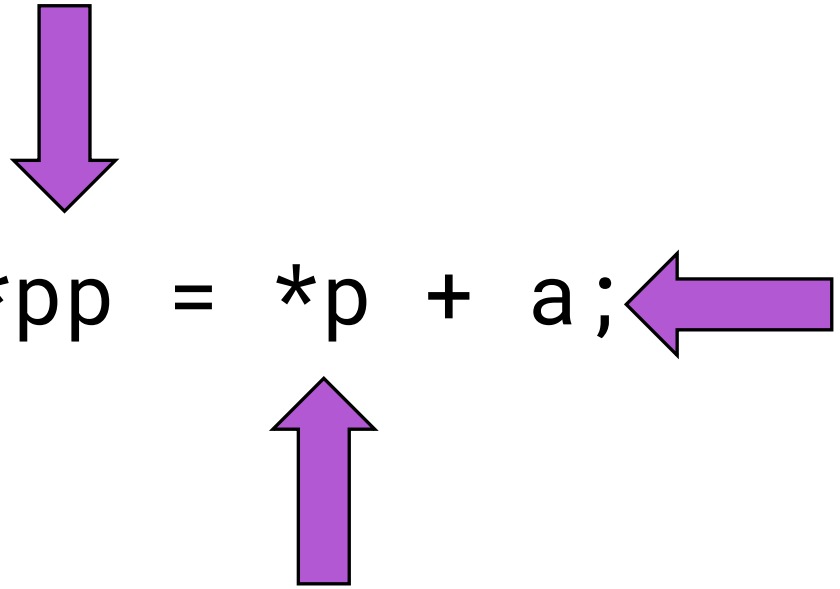
    printf("a = %d\n", a);
    printf("b = %d\n", b);
    printf("*p = %d\n", *p);
    printf("*q = %d\n", *q);

    return 0;
}
```

Memory diagram before `printf` is called

Double Pointers

Follow the arrow twice,
then write to that location.



```
**pp = *p + a;
```

Read the value at this location.

Follow the arrow once,
then read the value at
that location.